ShareCLI technical audit against operator intent

Audit date: 2026-10-14 Pacific. Evidence: read-only inspection of `/Users/kooshapari/CodeProjects/Phenotype/repos/sharecli`, HEAD `fec57c5ce45935203c63e5b4c487fddf642ff18c`. References below are repository-relative file and line anchors. Findings describe this local checkout, not a verified installed binary or deployed web instance.

Executive assessment

ShareCLI has substantial process pooling, host observation, thermal admission, IPC, coalescing and command-strategy machinery. The observed dashboard and detection contract remain centered on managed processes and recognized coding-agent families. This is insufficient for the operator's desired always-useful system optimization console. A zero-agent system should still expose process inventory, resource pressure, optimization opportunities and a usable policy catalog; an unknown harness should become an evidence-backed discovery candidate rather than disappear from the recognized-agent table.

The user-reported running web instance was not reproduced or captured. Source inspection found a credible dashboard delivery defect, definite misleading savings calculations in an external JavaScript file, a managed-pool/host-agent distinction that can make the empty state misleading, and a definite Windows spawn-core functional gap. No source was modified, tests/builds executed, runtime reconfigured, or process terminated.

Findings ordered by priority

ID	Priority	Finding	Evidence	Impact / required acceptance
A01	High	Native Windows spawn-core has no working process spawn/wait implementation and no inspected `wsl.exe` backend	`crates/spawn-core-sys/src/windows_stub.rs:1`, `:99`, `:112`; `build.rs:43`	Windows binary must implement native host process control and a separate WSL backend; semaphore compilation is not process-management parity.
A02	High	External dashboard JavaScript referenced without a matching HTTP route	`src/dashboard.html:808`; `src/commands/serve.rs:247-260`, `:415`	Source router serves HTML, image assets, health, config, metrics, profiling and WS; it has no `/dashboard.js` route or static fallback. Richer panels in the external JS are therefore unavailable through this router as inspected. Verify with a text-only HTTP route test before claiming deployed impact.
A03	High	Savings attribution conflates observed agent RSS with avoided memory	`src/dashboard.js:534`	`savings.saved_rss_bytes || agents.total_rss_bytes || null` reports current agent RSS as saved memory when savings are absent; zero is also discarded. Require measured avoided work or explicitly labeled forecast with baseline, interval, confidence and costs. This defect is in the externally referenced JS; A02 may prevent its execution.
A04	High	General-process policy path is missing in the inspected native strategy dispatcher	`crates/harness-native/src/dispatcher.rs:315-345`	Agent predicate runs before strategies; human/non-agent path directly executes real command. Existing strategies cannot establish all-process optimization from this path. Add independently scoped opt-in process classes and observation-only default.
A05	Medium	Empty managed pool directs user to start a harness, even though host observation is separate	`src/dashboard.html:464-478`; `src/commands/serve.rs:673-714`	`processes` comes from `ProcessPool.list`, while `agents` comes from `AgentProcSnapshot`. An empty managed list does not establish no harnesses on host. Show separate host observed, managed, recognized, candidate and observation-health counts.
A06	Medium	Detector is extensible through definitions but does not discover previously undefined families	`crates/sharecli-fleet/src/detect.rs:24-60`, `:67-89`; `proc_scan.rs:89-100`	TOML adds patterns without recompilation, but `scan_agents` only emits `match_known_agent` matches. No unknown candidate output exists in that scan contract. Add probabilistic evidence and operator review before classification/activation.

| A07 | Medium | Pool efficiency display computes busy slots and labels them idle | `src/dashboard.js:525-530`, `:542-549` | `total - idle` is summed as totalIdle, then used for idle/total and idle-slot labels. Correct the numerator and define usefulness independently from idle occupancy. Zero/absent data need explicit handling. |

| A08 | Medium | Inspected dashboard snapshot lacks typed savings and policy catalog fields | `src/commands/serve.rs:663-671` | Contract contains gate/host_watch/pool/status/agents/processes. The external JS optionally expects `data.savings`; this envelope supplies no top-level savings object. Add versioned policy/recommendation/measurement API rather than client inference. |

Existing capability inventory and fit

- Host detection: known families include Claude, Codex, Gemini, Cursor agent, Aider, Amp, Goose, Forge, Jcode and OpenCode. Runtime `agent_patterns.toml` definitions extend these. The configured names do not establish support for every alias/fork or unknown agent.

- Process trees: `sharecli-fleet/src/proc_scan.rs:103-146` walks ancestors and labels descendants; Linux uses `/proc`, other platforms use sysinfo (`:155-170`). Observation is broader than only ShareCLI-managed pool entries.

- Console transport: embedded HTML plus WebSocket. Existing gate, host-watch, pool, status and agent summaries form useful foundations. `serve.rs:1047-1121` contains operator-envelope tests, but this audit did not run them and serialization tests do not prove browser execution.

- Strategy execution: `harness-native/src/strategies/mod.rs:1-18` declares batch, causal order, circuit breaker, coalesce, debounce, hypervisor lane, incremental, jobserver, load balance, proactive warm, process, queue, resource throttle, retry and speculative modules. `RuleOpts` (`:25`) includes TTL, concurrency, cache keys, retry, causal domain, breaker and jobserver controls. Presence is implementation evidence, not measured effectiveness.

- Policy source: `harness-native/src/dispatcher.rs:110-145` loads `rules.conf`, resolves exact/subcommand and wildcard keys, and defaults to passthrough. This is an existing policy adapter to migrate into an inspectable versioned catalog, not a learned-optimization knowledge base by itself.

- Native spawn: POSIX Zig hot core with Rust FFI. Unix build invokes Zig (`spawn-core-sys/build.rs`); Windows bypasses Zig and provides real counting-semaphore behavior alongside unsupported process operations.

- Distribution: root `Cargo.toml:13-16` lists aarch64 macOS and x86_64 Linux metadata targets, with no Windows target there. Windows tray crates and tests exist, but their existence does not prove a released Windows management binary.

- Adjacent scope: mesh, sync, session, fleet, FUSE and tray crates exist. They are integration starting points; this audit does not establish complete cross-device peripheral/I/O/storage virtualization or cloud deployment control.

Interpretation of scheduling ambition

The inspected mechanisms are admission, pooling, launch policy, coalescing and resource observation. Unix `nice`, macOS `taskpolicy`, POSIX spawn and semaphore management are useful OS interfaces; they do not prove arbitrary syscall deduplication, a replacement kernel scheduler, hardware scheduling control, or optimization of every unrelated process. Treat each such ambition as a separate capability with platform, privilege, scope and benchmark evidence.

Safe semantic optimization requires explicit command equivalence, working directory, inputs, environment, tool version and side-effect classification. Never coalesce arbitrary mutations, interactive sessions, credential-bearing commands or non-deterministic work solely because command strings match. Learned recommendations should remain shadow proposals until negative controls establish preserved exit codes, output, effects and isolation. Existing strategy names alone do not satisfy those gates.

Native Windows and WSL audit requirements

The operator explicitly requires a Windows-native executable that manages WSL through `wsl.exe`; a WSL-installed Unix binary is an interim bridge only. Design `spawn-core` as a backend contract supporting native Windows host and

Windows-controlled WSL processes. Inventory installed/running distributions using bounded structured parsing of WSL CLI output; resolve distribution/user/path explicitly; use argument vectors and avoid shell-string interpolation. Separate Windows process ID, WSL distribution and Linux process ID, with lifecycle ownership and start identity to prevent PID reuse mistakes.

Acceptance must cover Unicode and space-containing paths, host/guest environment mapping, stdout/stderr and cancellation propagation, exact exit status, stopped/missing distribution behavior, denied elevation, distro shutdown/restart, nested children, deadlines and reconnect. Windows jobs may govern native Windows process trees; WSL guest process policy requires a guest capability contract. Do not report Windows job containment as guest kernel containment. Runtime proof must originate on a Windows host; macOS source review cannot qualify it.

Verification and remediation sequence

1. Establish installed daemon identity, binary hash/version, served HTML/JS route status and host scan health with CLI/HTTP text. Preserve the user's no-screenshot policy.

2. Add focused route/asset and empty-state contract tests in an authorized implementation lane. Cover empty managed pool with recognized host agents; zero agents with general work; inaccessible scan; disconnected daemon; unknown candidate; genuinely idle host.

3. Correct savings math and require typed measurement provenance. Unknown savings render unavailable, true zero remains zero. Never substitute allocated/current RSS for avoided work.

4. Expose the existing rule and strategy machinery as read-only catalog before enabling new policy mutation. Separate configured policy, active policy, proposed learning and measured result.

5. Introduce general-process observation and candidate discovery, then shadow optimization. Gate action by process ownership and command semantics.

6. Implement native Windows/WSL spawn-core backends and qualify on Windows hardware. Expand release target metadata only after runtime acceptance.

7. Link mesh compute/storage/I/O capability contracts to Fino Fabric and deployment intent to Byteport; avoid claiming adjacent product goals are implemented by a ShareCLI dashboard.

Evidence limits

No builds or tests were run: source audit was the requested output, the canonical checkout is already dirty, and builds would write into source-adjacent Zig caches. Test files are inventoried as evidence of intended verification only. No existing green result was rerun. GitHub PR, issue and Dependabot reads through `/opt/homebrew/bin/gh` failed with `error connecting to api.github.com`; current remote status remains UNKNOWN. No ChatGPT chat export, Cursor history, cloud chat, runtime daemon identity, installed binary version, performance baseline or numeric savings was verified in this technical lane. The parent report tracks broader source recovery.

Memory quick pass returned no relevant ShareCLI/Fabric/dream-loop registry matches, so no memory-derived technical claims are used. Source refs were observed in a live shared checkout and may move after this snapshot. No source commits/pushes/PRs were made, consistent with the explicitly read-only repository scope.

ShareCLI console and optimization system design

Status: proposed design, 2026-10-14 Pacific. This report changes documentation only. No implemented capability or measured saving is implied by a proposal below.

1. Product promise and current evidence

ShareCLI should improve the user's ability to get work done on a host, including when no coding harness is installed or running. Agent recognition supplies useful workload semantics; it must not gate observation, opportunities, strategy authoring, or optimization experiments. Fino Fabric extends this control and evidence model across compute, storage, I/O, and peripheral capabilities on multiple nodes. Byteport consumes deployment capabilities rather than making AWS the assumed destination.

Audit baseline: local ShareCLI HEAD `fec57c5ce45935203c63e5b4c487fddf642ff18c`. Repository findings supplied by the independent audit lane:

| Existing evidence | Consequence for design |

|---|---|

| `src/dashboard.js:23` prompts zero-pool users to start their first harness | Replace the dead-end with a working host optimization path. |

| `crates/sharecli-fleet/src/detect.rs` and `proc_scan.rs:89` use known agent patterns, including an extensible patterns file | Existing configuration extension is useful but is not autonomous discovery of previously unknown families. |

| `src/commands/serve.rs:247-255` registers the dashboard and APIs; the audit found no dashboard.js asset route despite `dashboard.html:808` referencing it | Consolidate asset delivery and test every supported entry path before relying on richer panels. |

| `src/dashboard.js:534` falls back from saved RSS to observed agent RSS | Remove this fallback. Current consumption is not saved consumption, and missing saving is not zero. |

| `dashboard.js:525-530` computes total minus idle and labels the result idle | Define and test metric semantics in a shared contract. |

| `crates/spawn-core-sys/src/windows_stub.rs:99-119` returns Unsupported; `build.rs:43-47` skips Zig on Windows | Native Windows management and a WSL backend are required implementation work. |

| Existing dispatcher rules and non-agent passthrough are reported in the audit | Do not market general process scheduling as already implemented. |

Exact source findings and their limitations belong to the companion audit. Proposed page names, APIs, data models, priorities, and tests below are new design decisions.

2. Information architecture

Use a persistent node selector, time window, observation freshness, and operating mode at the top. Modes are Observe, Simulate, Experiment, and Enforce. Changing nodes must never silently widen authority. The default landing page is Overview, independent of harness count.

| Page | Primary question | Main contents and actions |

|---|---|---|

| Overview | What is limiting my work, and what can I do now? | Host pressure timeline, foreground responsiveness, active work, ranked opportunities, experiment outcomes, missing capabilities. |

| Workloads | What is consuming resources? | All processes grouped by workload ancestry, known harnesses, unknown candidates, unmanaged applications, protected processes. |

| Opportunities | Where can performance improve? | Evidence, affected workload, proposed intervention, expected benefit range, cost, confidence, prerequisites, experiment action. |

| Rules and strategies | What behavior have I authorized? | Searchable strategy library, declarative rules, scope, conditions, exceptions, precedence, dry run and revision history. |

| Learned optimizations | What has actually worked here? | Validated findings, failed and inconclusive experiments, applicability envelope, recency, regression signals, promote or retire action. |

| Experiments and savings | What changed, and how certain is it? | Baselines, counterfactuals, actual outcomes, projected outcomes, uncertainty, regression guardrails, evidence export. |

| Nodes and capabilities | Where can work run or data live? | Local host, WSL distributions, VMs, remote hosts, storage-only providers, permissions, health and compatibility. |

| Activity and evidence | Why did the system act? | Ordered decisions, rule revisions, observations, action receipt, rollback receipt, redacted exports. |

| Settings and permissions | What may ShareCLI observe and control? | Sampling budget, retention, scope, allowlists, foreground protection, credential references, node trust and recovery. |

Keep the default screen comprehensible: one bottleneck narrative, three leading opportunities, and a link to full details. Advanced controls remain available in detail views. A global search finds workloads, strategies, evidence, and nodes. Tables support keyboard operation, explicit units, sortable freshness, and textual status alongside color. Text-based fixture tests and DOM/accessibility inspection satisfy the no-screenshot requirement.

3. First run and zero-harness behavior

Represent discovery and readiness separately. `0 recognized harnesses` may mean none are running, an adapter lacks visibility, permissions are restricted, discovery is stale, or detection does not recognize a family. Never turn these possibilities into a false claim that no agents exist.

First-run flow:

1. Identify host and capabilities using process/file/CLI observations. Show exactly what is visible and what is unavailable.

2. Start low-overhead observation within a declared collection scope. Record a baseline and collector cost; no process mutation is required.

3. Populate all-process Workloads and offer host-level opportunities with their evidence requirements.

4. Offer optional harness recognition and explicit workload registration. Continue when skipped.

5. Permit simulations and controlled experiments only where a supported actuator and authorized target exist.

Concrete initial screen text:

> **Your host is ready to observe**

> Recognized running harnesses: 0 ? Process scan: current ? Visibility: current user

> ShareCLI can still inspect resource pressure and suggest improvements. Recognition improves attribution as workloads appear.

> **Collect baseline** ? **Inspect workloads** ? **Review strategies**

> Savings: not measured. Estimates appear after relevant evidence is collected.

During an empty observation window:

> No sustained resource bottleneck observed in this window. Continue observing or select a workload to measure. No changes applied.

When visibility is incomplete:

> Harness count is incomplete: process details are unavailable for part of this host. Last successful scan: [time]. Inspect capability status.

When pressure exists but control is unavailable:

> Memory pressure is elevated. Evidence identifies [workload] as a contributor. This installation can observe it; the proposed control is unavailable with current permissions. Review a manual strategy or choose an owned workload experiment.

Zero harnesses must not produce fabricated opportunity counts, artificial savings, empty navigation, or a requirement to launch an agent. Capability-specific absence is displayed where it matters, not as a product-wide failure.

4. Open-world workload and harness discovery

Maintain an all-process inventory as the foundation. Identify an instance by node, boot/session identity, PID, and process start time to avoid PID-reuse mistakes. Associate executable identity, parent ancestry, owner, bounded launch metadata, and observed lifecycle. Redact arguments before persistence; never collect command secrets, environment dumps, chat contents, or credentials by default.

Detection is a layered classifier:

1. Known signatures: installed and running are separate states; match versioned executable/path/package/protocol signals.
2. Generic workload clusters: repeated CLI trees, child-tool launches, resource bursts, and known runtime relationships identify an *unknown workload candidate*, not a proven AI agent.
3. Evidence enrichment: inspect explicitly permitted installation metadata and manifests; do not run an unknown executable to learn what it does.
4. Local user mapping: identify or dismiss a candidate, register a custom executable, and preview matches before saving.
5. Optional reviewed adapter promotion: convert confirmed local evidence into a versioned adapter with positive and negative fixtures. Never silently upload local paths or process details.

Display candidate evidence and conflicting signals. Do not pretend a heuristic score is a calibrated probability. Distinguish recognized agent, user-labeled agent, unknown CLI workload, generic application, and excluded process. Multiple signals derived from the same ancestry count as correlated evidence. A shell running `node` alone is insufficient to identify an agent.

Adapters declare supported OS, signature version, evidence requirements, confidence semantics, and test fixtures. New adapter versions first run in shadow mode. Negative controls include editors, ordinary shells, language servers, package managers, build tools, renamed executables, wrappers, and synthetic PID reuse. Missed candidates remain visible as general workloads.

5. Opportunity engine and strategy library

Pipeline: Observe -> normalize -> attribute -> find constraints -> propose -> estimate -> simulate -> experiment -> evaluate -> learn -> optionally enforce. Each stage has a version and an evidence receipt. An empty or failed stage produces a reason, not a guessed result.

Candidates can arise from resource contention, duplicated work, excessive launch churn, redundant polling, avoidable rebuilds, locality problems, scheduling conflicts, repeated failures, or unsuitable placement. These are hypotheses until a relevant observation supports them. Rank by user objective, expected net benefit, evidence quality, action reversibility, prerequisite cost, and regression risk. Show Pareto alternatives when throughput and responsiveness conflict; never reduce every outcome to a misleading universal score.

| Strategy family | Preconditions | Intervention concept | Required outcome and guardrail |

|---|---|---|---|

| Concurrency shaping | Independent queued jobs, known ownership | Bound concurrent builds or tool launches | Completion time/throughput and queue delay; no starvation. |

| Foreground protection | Explicit foreground preference and controllable background work | Lower background scheduling priority or pause new admission | Interaction latency plus background deadline compliance. |

| Launch/poll coalescing | Repeated equivalent operations with understood semantics | Coalesce safe requests or adjust opt-in polling cadence | Spawn/syscall count per completed task plus freshness bound. |

| Shared immutable cache | Content identity and cache validity proven | Reuse build/download results | Avoided work plus correctness and storage costs. |

| Memory admission | Repeated peak-memory contention | Queue owned jobs before pressure becomes critical | Peak memory, stall duration and completion time; no forced data loss. |

| I/O locality | Measured read/write behavior and eligible target | Place scratch/data near execution | I/O wait and total transfer cost; integrity checks. |

| Cross-node placement | Fabric capability, trusted target and portable workload | Move eligible queued work | End-to-end time including transfer, startup and return. |

| Adapter optimization | Known harness supports a documented setting | Recommend or apply scoped setting | Versioned behavior and task-quality regression checks. |

Avoid promising generic syscall deduplication: syscalls carry ordering, ownership, and side-effect semantics. Initial coalescing belongs at explicit application/tool boundaries. OS scheduling integration is a capability layer with platform-specific mechanisms; hardware scheduler control is research unless a documented supported interface is proven.

6. Rule editor and learned optimization pages

A rule contains an ID, revision, owner, intent, target selector, trigger, evidence threshold, capability prerequisites, action parameters, maximum scope, exclusions, cooldown, budget, guardrails, rollback recipe, expiry, and audit links. Render human-readable logic and an advanced declarative view from the same model.

Example proposed rule:

> When this node shows sustained memory pressure and more than the configured number of **owned build jobs** are queued or running, stop admitting additional builds. Keep foreground and explicitly protected workloads exempt. Resume admission after the configured recovery window. Never kill an existing process. Preview shows the exact selected jobs and exclusion reasons.

Resolve conflicts deterministically: protected scopes and explicit deny take precedence, then authority scope, then documented priority and specificity. Reject contradictory equal-precedence mutations rather than relying on evaluation order. Show a rule explanation for every decision. Rules are distinct from harness configuration files; changes to shared harness policy require the established all-three-harness update workflow.

A strategy is a reusable hypothesis/template. A rule is an authorized executable policy. A learned optimization is a contextual finding backed by experiments. Promotion between these concepts is explicit.

Learned findings show workload signature, machine/OS envelope, strategy version, tested conditions, sample size, effect range, confidence method, regression outcomes, freshness, provenance, and supersession. Failed experiments remain searchable to prevent repeated mistakes. New OS, hardware, adapter or workload versions can invalidate applicability. Promotion to enforcement requires a valid rollback and policy authorization; learning alone grants no control authority.

7. Measurement and honest savings

Use four labels everywhere: **observed**, **estimated**, **experiment result**, and **not available**. A model estimate never fills an observed-savings field. Missing values remain null with a reason. A zero value is valid data, so truthiness fallbacks are prohibited.

Metrics include completed useful tasks, end-to-end wall time, queue delay, interaction latency, CPU time, peak and integrated memory, pressure/stall duration, bytes transferred, I/O latency, spawn count, supported syscall counts, energy where measured, and actual monetary cost where a price and accounting basis exist. Track result quality and errors alongside speed. Lower RSS alone can mean the workload was delayed or failed.

Platform contracts:

| Platform | Native evidence | Explicit limitations |

|---|---|---|

| Linux | Process counters, cgroup metrics where delegated, CPU/memory/I/O PSI where enabled | PSI is stall time, not CPU utilization; unsupported kernel/cgroup data stays unavailable. |

| macOS | Process CPU/RSS, OS memory-pressure indicators, swap/compression counters where available, workload latency, supported disk metrics | Do not rename memory-pressure categories as Linux PSI or sum category values into a synthetic PSI percentage. Detailed syscall tracing needs a separate supported/permissioned collector. |

| Windows | Native process counters, commit/working-set measurements, paging and CPU/disk counters where available, workload latency | Commit, working set, and physical memory are different measurements. Native Job Object control does not establish control of Linux descendants inside WSL. |

| WSL | Per-distribution Linux evidence collected through a scoped backend plus Windows host evidence | Keep guest and host namespaces separate; avoid double counting VM memory/CPU and guest process totals. |

Linux PSI measures CPU, memory and I/O stalls and may expose per-cgroup pressure with appropriate configuration. See [Linux kernel PSI documentation](https://docs.kernel.org/accounting/psi.html).

Illustrative formulas, with no verified numeric savings claimed:

- Wall-time saving per comparable job: `T_baseline - T_treatment`; relative saving: `(T_baseline - T_treatment) / T_baseline`, only when baseline is positive.

- Projected time saving over a horizon: `eligible_job_count ? estimated_effect_per_job ? additional_setup_and_transfer_time`. Mark eligibility and effect assumptions separately.

- Memory residence reduction: `integral(M_baseline(t) ? M_treatment(t), dt)` in GiB-seconds, on comparable completed work. Peak RSS reduction is a separate number.

- Stall reduction: `stall_seconds_baseline ? stall_seconds_treatment`, with absolute percentage-point and relative changes clearly distinguished if displaying pressure ratios.

- Net remote benefit: `local_completion_time ? (queue + transfer_out + remote_execution + transfer_back + startup)`. Include remote failure probability and retransfers in uncertainty.

- Projected cost saving: `baseline_billable_units ? baseline_rate ? treatment_billable_units ? treatment_rate ? added_costs`. Idle capacity is not cash saved without a real avoidable cost.

- Collector net benefit: `gross_effect ? observation_and_control_overhead` in the same unit. Do not subtract unlike units.

Never add overlapping savings from two rules affecting the same task. Deduplicate attribution by experiment and workload/time window; show joint effects or marginal estimates explicitly. Time projections do not imply the user gained equivalent working time unless the work was on their critical path.

Measurement protocol: capture task identity, input digest, completion semantics, relevant node state and strategy version; establish a comparable baseline; randomize or alternate matched windows where safe; account for warm caches, load, thermal effects, network variation and learning effects; include an untreated negative control; predeclare success and regression thresholds; report sample count and interval method. If samples are insufficient or workload drift is material, show inconclusive. For nonrepeatable interactive work, clearly label observational evidence and confounding.

Confidence ranges must come from an identified model or statistical method. Estimate calibration compares forecast intervals with later results, tracks coverage and directional errors, and suspends unreliable models. A single-user low-sample history should not produce precise-looking percentages.

8. Capability and control boundaries

Start with observation and application-level admission control for owned work. Then integrate documented OS primitives, validated per OS/version and permission level. Linux cgroup controls require appropriate delegation; Windows may use supported native process/Job Object interfaces; macOS policy mechanisms require independent feasibility verification. The UI only offers an action when its capability probe is positive.

Windows Job Objects can manage process groups and CPU-rate controls, subject to documented nesting and breakaway behavior. They are not a universal cross-OS scheduler. See [Job Objects](https://learn.microsoft.com/en-us/windows/win32/procthread/job-objects) and [CPU rate control](https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-jobobject_cpu_rate_control_information).

Every mutation revalidates identity and target ownership immediately before acting. Keep protected/system processes excluded unless explicitly supported and separately authorized. Provide global pause, per-rule disable, bounded duration, rollback, offline recovery and a readable audit receipt. An unavailable collector must disable dependent decisions, not preserve stale enforcement as though evidence were current. Prevent oscillation with hysteresis, cooldowns and action budgets.

Do not automatically kill rustc/cargo or other shared processes as an optimization. Do not silently broaden collection, install tracing components, change global harness settings, or alter remote trust. None of those is necessary for zero-harness value.

9. Native Windows and WSL backend in spawn-core

Required end state: a Windows-native ShareCLI executable observes and manages its Windows host and invokes WSL through `wsl.exe` using a WSL backend in spawn-core. A Linux service reached through an interim bridge does not meet this requirement.

Proposed backend interface: `probe`, `list_targets`, `spawn`, `status`, `wait`, `cancel`, `read_output`, and `capabilities`. Execution target is a tagged value (`WindowsHost` or `WslDistribution{id,name,user}`); never infer target from path shape. Each spawn request includes an argument vector, explicit working directory, scoped environment overrides, stdio mode, deadline, cancellation semantics, and idempotency key. Return an opaque job identity with host launcher identity and guest process identity when established.

Use a trusted system `wsl.exe` path, supported distribution selection and direct execution options. No generic shell-string interpolation. Validate argument encoding at the Windows process boundary and again at guest execution. Probe actual supported command behavior instead of assuming a particular WSL version. Official command surface: [Microsoft WSL command reference](https://learn.microsoft.com/windows/wsl/reference).

A Windows launcher PID is not a Linux workload PID. Track guest process start identity with a helper/protocol where required. Define whether cancellation terminates only the launched guest job and descendants; do not terminate a whole distribution to cancel one task. If cancellation cannot be established, mark it unsupported and reject workloads requiring it. Propagate output, exit status, timeout and partial failure distinctly. Native spawn and WSL spawn can coexist.

Required contract and integration tests:

1. Native executable starts with WSL absent: Windows observation works, WSL capability is unavailable, no Linux-only dependency aborts startup.

2. Enumerate multiple distributions, including stopped ones and Unicode names; enumeration does not start a stopped distribution without an explicit launch request.

3. Target selection launches only the requested distribution/user; changing the system default does not redirect an existing configured target.

4. Round-trip argument vectors containing spaces, quotes, Unicode, shell metacharacters, and empty strings; arguments are never executed as injected commands.

5. Verify cwd and path conversion for Windows and guest paths; reject unsupported mappings rather than guessing.

6. Environment overrides are scoped to the job; secrets do not enter telemetry or error text.

7. Capture stdout/stderr and exact nonzero exit results; distinguish transport failure from child failure; bound buffering and handle backpressure.

8. Cancellation and timeout affect only the owned workload subtree; unrelated Windows and WSL jobs survive. Verify guest cleanup, not merely launcher exit.

9. Handle guest exit before registration, Windows PID reuse, launcher crash, distribution shutdown and backend restart without attaching to an unrelated process.

10. Text-only native Windows/WSL integration run proves lifecycle behavior on a real Windows host; cross-compilation alone is insufficient.

10. Fabric and Byteport composition

Fabric nodes publish separate executable, storage, I/O and peripheral capabilities. Administrator access supports a full host role only after capability probes and trust establish what can actually run. A storage-only service is a storage provider, not a compute node. A restricted client can be an access endpoint without being eligible for arbitrary remote execution.

ShareCLI supplies local workload intent, pressure evidence, safe admission/control and measurements. Fabric supplies authenticated placement, transport, inventory, leases and cross-node recovery. Byteport supplies build/deploy intent, dependency/artifact contracts and provider adapters. Shared job IDs and capability schemas enable composition without collapsing product ownership. Optimize data movement and total completion time rather than merely distributing processes. Peripheral access requires explicit ownership, contention policy and device semantics; it is not equivalent to mounting disk.

11. Proposed data and API contracts

Version all envelopes. Store UTC event time, monotonic duration where possible, collector version, node identity and freshness. Redacted exports contain evidence references and hashes, not undisclosed secrets.

| Entity | Minimum contract |

|---|---|

| Capability | node, platform, feature, available/unsupported/denied/unknown, observed_at, reason, probe version |

| Workload | stable process identity, ancestry, recognition class, adapter version, ownership, redacted display, visibility |

| Observation | metric, value or null, unit, scope, time interval, provenance, quality, missing reason |

| Opportunity | hypothesis, evidence references, affected scope, prerequisites, proposed action, estimate model/version, range, costs, conflicts |

| Rule revision | declaration, owner, authority scope, precedence, protection, guardrails, rollback, expiry, dry-run receipt |

| Experiment | assignment, baseline/treatment, task identity, thresholds, outcomes, confounders, analysis version, conclusion |

| Learned finding | experiment references, applicability, effect interval, confidence method, recency, promotion state |

| Action receipt | idempotency key, identity revalidation, rule revision, before state, action, result, rollback status |

Suggested read APIs: `/api/v1/capabilities`, `/workloads`, `/opportunities`, `/strategies`, `/rules`, `/findings`, `/experiments`, `/savings`, `/events`. Mutating APIs require explicit policy scope, revision concurrency checking and idempotency. Stream typed, sequence-numbered updates with a resynchronization endpoint; a dropped connection renders data stale and cannot silently preserve current status. These endpoints are proposals, not existing routes.

12. Phased backlog and objective acceptance

No fabricated schedule or savings target is assigned. Each phase depends on evidence from its predecessor; independent UI/data-contract work can proceed in parallel with OS-adapter work.

| Phase | Deliverable | Objective acceptance and negative control |

|---|---|---|

| A: truthful baseline | Fix asset delivery, idle labels and savings semantics; separate discovery/visibility/capabilities | Server entry test loads every declared asset; zero and null contract fixtures remain distinct; observed RSS never appears as saved RSS; idle+busy reconciles to denominator. |

| B: zero-harness console | All-process overview, baseline collection, opportunity empty states and capabilities | Fixtures for zero processes, zero known harnesses with busy generic processes, restricted visibility and stale scans all retain useful navigation; no harness launch is required. Collector overhead is measured against a predeclared budget. |

| C: discovery | Generic candidates, adapter registry, local registration and review workflow | Unknown synthetic workload appears as a candidate; known fixtures match expected adapters; ordinary shell/editor/build negative controls are not asserted to be agents; PID-reuse fixture cannot transfer identity or authority. |

| D: rules and experiments | Strategy library, rule preview, reversible admission-control trial, findings ledger | Dry run lists exact affected/excluded identities; conflicting policies reject safely; no-op negative control reports no causal saving; rollback restores controlled state; failed/inconclusive results remain visible. |

| E: calibrated savings | Comparable baselines, explicit estimates, outcome intervals and deduplicated attribution | Known synthetic effects reproduce expected unit conversions; overlapping effects are not summed twice; unavailable/negative effects render correctly; calibration evaluates held-out outcomes and displays insufficient evidence honestly. |

| F: native Windows | Native Windows process backend and spawn-core WSL backend | All ten native/WSL lifecycle contracts above pass on real Windows; build-only or bridge-only evidence cannot close the phase. |

| G: deeper scheduling | Supported Linux/macOS/Windows controls behind capability gates | Every enabled actuator passes ownership, permissions, stale-identity, starvation, oscillation and rollback tests on its target OS; unsupported hardware claims remain research. |

| H: Fabric/Byteport | Typed capability integration and one bounded cross-node placement experiment | Storage-only node is rejected for execution; transfer/startup/return included in outcome; disconnect and lease-expiry recovery preserve workload ownership; local fallback is explicit. |

Product gate across phases: user-visible opportunities link to evidence; estimates show assumptions and uncertainty; every action explains authority and rollback; a clean host with no recognized harness still has a useful console. Performance acceptance uses completed work and responsiveness jointly, not a single resource counter. Local tests, native runtime

qualification, hosted CI, deployment and economic outcomes remain separate evidence gates.

13. Immediate implementation handoff

Begin with Phase A and B against the exact audited revision after repository-state reconciliation. Freeze the measurement vocabulary and capability schema before building savings cards. Then implement candidate discovery and one reversible, owned-work admission strategy to prove the full learning loop. Develop native Windows/WSL lifecycle support as a named independent lane sharing the same job contract. Do not begin broad autonomous OS intervention until those foundations are measured and recoverable.

This document is a console and optimization specification. It does not approve production deployment, mutate harness configuration, establish measured gains, or substitute for unavailable chat history and native Windows runtime evidence.

ShareCLI / Fino Fabric dream loop - registry-ready proposal

Status: PROPOSED; 2026-10-14 Pacific. This is a planning artifact, not a change to PhenoRegistry, shipped behavior, an approved kernel intervention, or measured savings. Exact operator brief is preserved in `operator-brief.md`.

Intent and ownership

ShareCLI owns local software scheduling and resource supervision across all processes: agent adapters enrich identity but do not gate usefulness. Fino Fabric extends local resource capabilities into a user-oriented mesh of hosts, VMs, compute, storage, peripherals and I/O. Byteport consumes those capabilities to build and deploy projects from declared repository paths and optional explicit commands. Providers may include self-hosted nodes, Vercel and Supabase; AWS is optional rather than the default economic assumption.

A full host advertises granted execution/control capabilities; administrator access alone does not imply trust or an automatic grant. A client may only request access. A storage-only provider such as Google Drive advertises storage operations and quotas, never executable capacity. Peripherals and I/O require exclusive/shared lease semantics, device ownership, latency budgets and loss recovery. Do not pretend transparent distributed memory or arbitrary GPU pooling follows from networking nodes.

Canonicalization evidence and unresolved history

Current local registry `handbook/specs/program-docset/products/ShareCLI/DOSSIER.md` calls ShareCLI an OS-adjacent process/resource supervisor and requires correctness, compatible coalescing, comparative measurement and negative controls. Its `FEDERATION.md` proposes two hosts sharing a permitted compatible instance. BytePort/FEDERATION.md proposes deployment/status/rollback contracts and scoped process supervision. These support the boundaries above. Older `ECOSYSTEM_MAP.md` and rationalization records describe archived/absorbed projects; those historical dispositions conflict with newer product records and must not authorize deletion or source movement.

No exact dream-loop definition was recovered from the bounded local registry Markdown search. The phrase therefore has no verified existing canonical semantics in this audit. Adopt the definition below as a proposal, retain a supersedes field for any subsequently recovered source, and attach the recovered October 12 chat snapshots before claiming historical continuity. The available October 4 Plan 100x Revenue export is a different conversation ID from the requested active Branch: [Platform] chat; it does not prove that active branch was recovered.

Proposed canonical definition

A dream loop is a bounded, evidence-producing cycle which converts desired user outcomes and observed friction into ranked hypotheses, executes one reversible experiment, independently checks correctness and value, and persists either a scoped optimization or a rejected hypothesis with its evidence. Each cycle has a workload, owner, budget, stop condition, baseline, counterfactual, provenance and rollback. Ideation may be broad; enactment remains limited to granted capabilities.

State machine: intake -> inventory -> model -> rank -> simulate -> experiment -> verify -> decide -> learn -> observe. Decisions: promote, retain as suggestion, reject, pause for missing evidence, or rollback. A failed hypothesis is useful learned evidence; repeated identical retries without a new hypothesis are not progress.

Record schema: id, revision, source_refs, product_boundary, outcome, owner, capability_grants, workload_signature, hypothesis, constraints, baseline_revision, candidate_revision, experiment_budget, metrics, negative_controls, projected_effect_range, measured_effect, confidence_method, external_validity, acceptance_gate, stop_reason, rollback_receipt, learned_rule_refs, supersedes, next_review_trigger. Financial metrics require actual invoices/receipts or

explicit estimates and never equate avoided CPU time with revenue.

Delivery DAG and acceptance gates

| Work package | Depends on | Deliverable and exit evidence |

|---|---|---|

| DL-01 Source recovery | none | Immutable brief, source register, selected chat excerpts with IDs/timestamps and access status; unresolved active branch clearly marked |

| DL-02 Capability atlas | DL-01 | Actual code/build/runtime map, source ownership and supported OS capabilities; baseline and unknowns |

| DL-03 Zero-harness console | DL-02 | System workload overview, rules and strategies usable with zero recognized harnesses; fixture proves absent, permission-denied, stale and empty are distinct |

| DL-04 Discovery | DL-02 | Explainable unknown workload candidates; passive metadata first; adapter proposal and operator override; false-positive and spoofing controls |

| DL-05 Rule ledger | DL-03 | Versioned rule strategies, simulation, permission scope, provenance, enable/disable and rollback; learned optimization lifecycle |

| DL-06 Controlled experiments | DL-05 | Baseline vs candidate with independently held negative cases, same workload, counterbalanced order, repeated samples, confidence bounds and overhead |

| DL-07 Windows native | DL-02 | Windows PE binary, native host discovery/control and spawn-core WSL backend using wsl.exe; lifecycle/cancellation/native isolation evidence |

| DL-08 Fabric contract pilot | DL-06, DL-07 | Two granted hosts plus storage-only provider; capability negotiation, leases, partition behavior and revoke/reconnect proof |

| DL-09 Byteport journey | DL-08 | One project/subrepo build and deployment plus status/rollback observed from both applications without duplicate action |

| DL-10 Registry handoff | all scoped gates | Additive pattern/ADR, ownership map, linked work packages and evidence index; source owner accepts explicit unresolved items |

No invented durations. Estimate each package only after inventory exposes its denominator; report owner, elapsed time, evidence and revised ETA when execution begins.

Windows is a required native product surface

Ship a Windows executable with native host process/resource telemetry and control. Model WSL as a spawn-core backend selected by a structured target containing distribution, Linux user, working directory and argument vector. Invoke wsl.exe through a direct process API rather than concatenated shell strings. Discover installed distributions and version/status; keep Windows and Linux process identity namespaces separate. Detect missing WSL, disabled virtualization, stopped/missing distributions, denied rights and unsupported versions explicitly.

The Windows parent must define stream decoding, Windows/WSL path conversion, quoting, environment allowlists, exit codes, launch timeouts, cancellation escalation, descendant cleanup and reconnect behavior. Killing only wsl.exe is not proof the Linux workload stopped: use a distribution-side scoped supervisor/job token with a verified termination receipt. Native Windows jobs and WSL jobs require separate implementations and integration tests. Do not terminate a whole distribution to cancel one task. Test Unicode/spaces, UNC paths, concurrent distributions, parent crash, Linux grandchildren, output backpressure and client disconnect. A WSL-hosted bridge may remain interim but cannot close DL-07. Microsoft documents Windows-to-WSL command execution through wsl.exe: https://learn.microsoft.com/en-us/windows/wsl/basic-commands and https://learn.microsoft.com/en-us/windows/dev-environment/wsl-interop .

Optimization sequence and feasibility boundaries

Start with admission control, bounded concurrency, shared read-only work under proven equivalence, cache scoping, polling backoff and priority-aware queues. Progress to OS-specific supported controls only with capability checks and measured benefit. Kernel/syscall or hardware scheduling changes are research lanes requiring an explicit platform mechanism, compatibility analysis, privileges, isolated benchmark and rollback; user-space process deduplication does not establish syscall coalescing. Never merge side-effectful operations solely because command text matches. Credentials, inputs, environment, working directory, tool versions, authorization subject, filesystem state and cancellation consumers affect equivalence.

Zero-harness operation still offers process groups, pressure indicators, optimization opportunities, a dry-run schedule and rule history. Unknown executable metadata produces an unclassified workload candidate, not an invented agent identity. Discovery must distinguish installed, running, connected and controllable states. Read no chat content or secrets merely to classify a workload.

Measurement and savings contract

Every forecast shows workload, baseline, assumptions, sample window, confidence and expiration. Keep wall time, CPU-seconds, resident/private memory, peak memory, I/O bytes, queue delay and platform pressure metrics separate. Linux PSI, macOS pressure and Windows counters need native definitions and must not be displayed as one interchangeable percentage.

Example arithmetic only: if baseline work consumes 120 CPU-seconds, 30 are proven redundant, and supervision costs 3, projected net reduction is 27 CPU-seconds or 22.5%. This is not a measured ShareCLI result and implies no fixed wall-time improvement. Memory savings require simultaneous lifetime accounting; summing process RSS double-counts shared pages. Cross-host placement evaluates local compute + remote transfer + remote queue + remote execution + result transfer against the local baseline; storage/cache costs and energy are separate. Additive savings cannot sum overlapping rule opportunities; evaluate the combined schedule.

Guardrails: correctness and authorization invariants first; no foreground p95/p99 regression above a predeclared workload threshold; observer overhead bounded before promotion; latency/error/leak negative controls; rollback on breach; kill switch and versioned policy. Report measured benefit with uncertainty and inconclusive outcomes.

External methods and skills adopted as guidance

Brendan Gregg's USE method motivates resource-by-resource utilization, saturation and errors rather than attractive but ungrounded dashboards: https://www.brendangregg.com/usemethod.html . Anthropic's long-running harness account motivates persistent task state and small tested increments: https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents . These are design inputs, not product proof. The local continuous-agent-loop skill recommends bounded loops, quality/eval gates, persistence and recovery from churn; its guidance informs the proposal without invoking an actual execution loop. Any external screenshot-based verification suggestions are excluded by the operator policy; verification here uses text and process-scoped evidence.

Proposed registry landing, without mutation

Propose `patterns/dream-loop.md` as the reusable method; add product-specific links under the current ShareCLI dossier, a Fino Fabric boundary proposal, and BytePort federation/deployment records. Reconcile `BOUNDARY_OWNERS.md` and historical archive aliases before registering ownership. Use the registry session artifact protocol and link an AgilePlus specification during later implementation; this audit does not claim live engine state or create that spec. Preserve source IDs and hashes, add a claim-to-evidence matrix, document conflicts and define explicit supersession rather than deleting older records.